Firmware Architecture Draft - Week 2, Monday

Likhitha N | IoT / Embedded Intern | Crash Connect Pvt Ltd

Draft based on my own understanding of the system architecture (vehicle, ESP32-S3, GNSS, IMU, LTE, SOS processing, MQTT/API, backend). Items marked **Pending Confirmation** need team confirmation.

1. Firmware modules

Module	Responsibility
GNSS	Reads NMEA sentences from the GPS module, parses them, and checks that the fix is valid and recent.
IMU	Reads accelerometer and gyroscope data and computes values such as acceleration magnitude.
Event / SOS logic	Decides whether motion data looks like a crash and raises an event. **Pending Confirmation**: thresholds, manual SOS button.
Telemetry builder	Packages location, motion and status into one message with device ID and timestamp. **Pending Confirmation**: exact message format.
Communication	Sends messages through the LTE modem to the backend, with retries. **Pending Confirmation**: MQTT or API, modem type, UART/AT interface.
Device status / health	Tracks what is working (GPS fix, sensors, network) and reports it.
State machine	Holds the current device state and decides which transitions are allowed.
Logging / storage	Keeps debug logs and any buffered messages. **Pending Confirmation**: whether offline buffering is needed.

2. State transitions (draft)

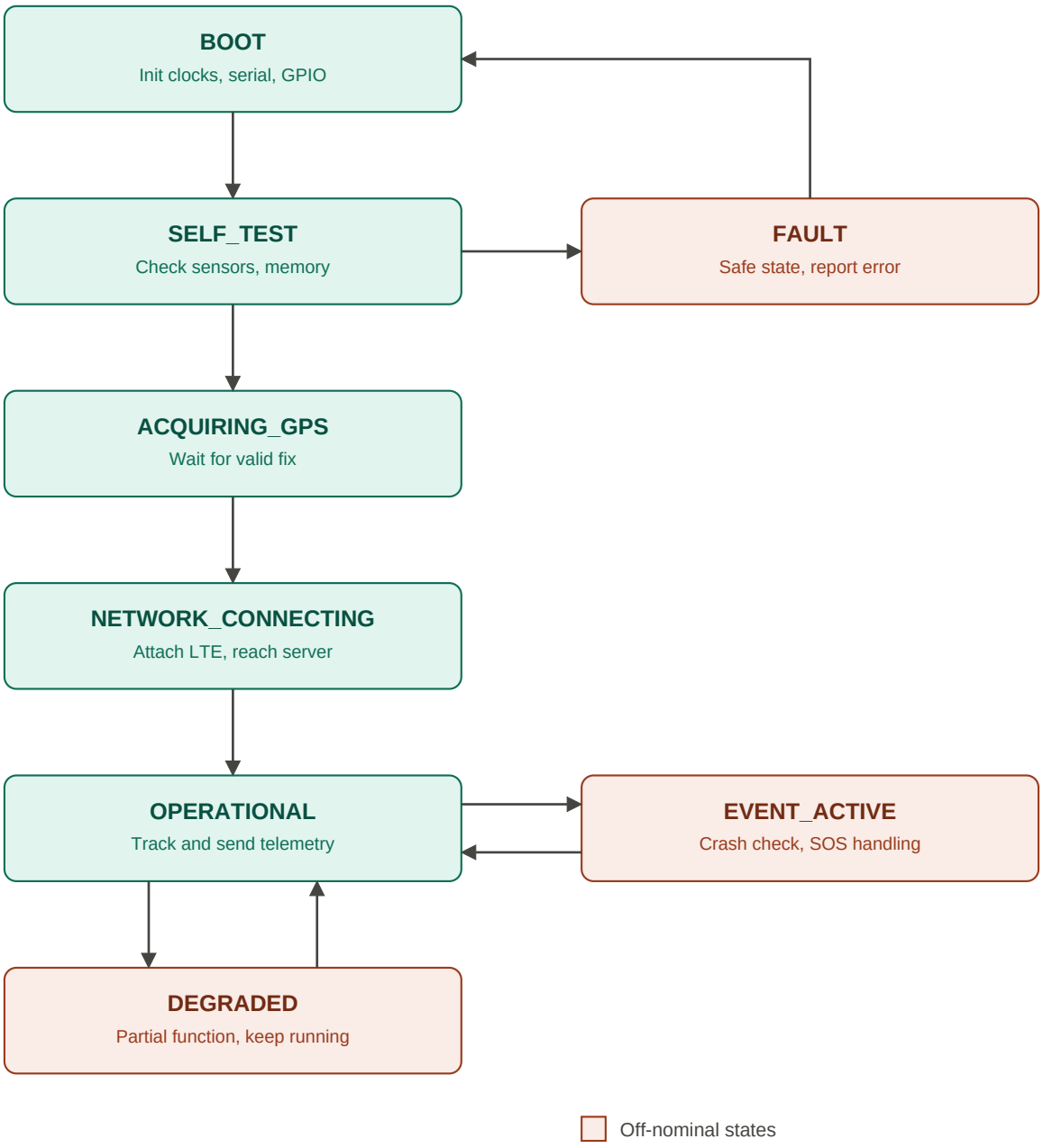
From	To	Trigger
BOOT	SELF_TEST	Clocks, serial and GPIO initialised
SELF_TEST	ACQUIRING_GPS	Sensors and memory checks pass
SELF_TEST	FAULT	Critical check fails. **Pending Confirmation**: automatic retry?
ACQUIRING_GPS	NETWORK_CONNECTING	Valid GPS fix obtained. **Pending Confirmation**: or run in parallel with network?
NETWORK_CONNECTING	OPERATIONAL	LTE attached and server reachable
OPERATIONAL	EVENT_ACTIVE	Suspected crash or SOS request
EVENT_ACTIVE	OPERATIONAL	Event sent and cleared
OPERATIONAL	DEGRADED	Partial failure, e.g. GPS fix or network lost
DEGRADED	OPERATIONAL	Failed part recovers
FAULT	BOOT	Reset

3. Pending Confirmation

1. Does the device wait for a GPS fix before connecting to the network, or do both run in parallel?
2. Can any state go straight to FAULT, or only SELF_TEST?
3. Does EVENT_ACTIVE interrupt everything, or run alongside normal tracking?
4. Which failures lead to DEGRADED instead of FAULT?
5. Is a manual SOS button part of the design?
6. Is the LTE modem connected over UART, and is the transport MQTT or a REST API?

Note: the diagram on the next page shows the main paths only. Other arrows (for example DEGRADED to FAULT) are part of the Pending Confirmation list.

4. Device-state diagram (draft)



Teal = normal start-up path. Coral = off-nominal states. Arrow from FAULT back to BOOT = reset.